

复习课速记

10 客观 (2s): 选择, 判断

判断: 有环必死锁, 死锁必有环

怎么解决 cpu 传递速度与打印机打印速度不匹配, 什么技术 (通道? 虚拟?)

10 简答 (5s)

绪论

进程模型, 线程模型, 还有什么执行模型, 列举

互斥怎么办

系统调用, 原理

进程间互斥死锁调度

存储管理, 基本存储管理, 虚拟存储管理, 页面调度算法, 页面机制原理, 缓存

文件系统概念, 物理, windows, fat, 原理, 计算

io 访问方法, 特点, 存储, 磁盘调度 (原理计算)

可靠存储, 方法原理

视窗系统

电池管理

计算&程序(30 分)

程序: 多线程, 系统调用, 死锁, 进程间互斥&通讯

用户级线程切换, 劫持

怎么输出 helloworld 互不干扰

对于某些应用而言，决不丢失或破坏数据是绝对必要的，即使面临磁盘和CPU错误也是如此。理想的情况是，磁盘应该始终没有错误地工作。但是，这是做不到的。所能够做到的是，一个磁盘子系统具有如下特性：当一个写命令发给它时，磁盘要么正确地写数据，要么什么也不做，让现有的数据完整无缺地留下。这样的系统称为**稳定存储器 (stable storage)**，并且是在软件中实现的 (Lampson和Sturgis, 1979)。目标是不惜一切代价保持磁盘的一致性。下面我们将描述这种最初思想的一个微小的变体。

在描述算法之前，重要的是对于可能发生的错误有一个清晰的模型。该模型假设在磁盘写一个块（一个或多个扇区）时，写操作要么是正确的，要么是错误的，并且该错误可以在随后的读操作中通过检查ECC域的值检测出来。原则上，保证错误检测是根本不可能的，这是因为，假如使用一个16字节的ECC域保护一个512字节的扇区，那么存在着 2^{4096} 个数据值而仅有 2^{144} 个ECC值。因此，如果一个块在写操作期间出现错误但是ECC没有出错，那么存在着几亿个错误的组合可以产生相同的ECC。如果某些这样的错误出现，则错误不会被检测到。大体上，随机数据具有正确的16字节ECC的概率大约是 2^{-144} 。该概率值足够小以至于我们可以视其为零，尽管它实际上并不为零。

该模型还假设一个被正确写入的扇区可能会自发地变坏并且变得不可读。然而，该假设是：这样的事件非常少见，以至于在合理的时间间隔内（例如1天）让相同的扇区在第二个（独立的）驱动器上变坏的概率小到可以忽略的程度。

该模型还假设CPU可能出故障，在这样的情况下只能停机。在出现故障的时刻任何处于进行中的磁盘写操作也会停止，导致不正确的数据写在一个扇区中并且后来可能会检测到不正确的ECC。在所有这些情况下，稳定存储器就写操作而言可以提供100%的可靠性，要么就正确地工作，要么就让旧的数据原封不动。当然，它不能对物理灾难提供保护，例如，发生地震，计算机跌落100m掉入一个裂缝并且陷入沸腾的岩浆池中，在这样的情况下用软件将其恢复是勉为其难的。

稳定存储器使用一对完全相同的磁盘，对应的块一同工作以形成一个无差错的块。当不存在错误时，在两个驱动器上对应的块是相同的，读取任意一个都可以得到相同的结果。为了达到这一目的，定义了下述三种操作：

1) **稳定写 (stable write)**。稳定写首先将块写到驱动器1上，然后将其读回以校验写的是正确的。如果写的不正确，那么就再次做写和重读操作，一直到 n 次，直到正常为止。经过 n 次连续的失败之后，就将该块重映射到一个备用块上，并且重复写和重读操作直到成功为止，无论要尝试多少个备用块。在对驱动器1的写成功之后，对驱动器2上对应的块进行写和重读，如果需要的话就重复这样的操作，直到最后成功为止。如果不存在CPU崩溃，那么当稳定写完成后，块就正确地被写到两个驱动器上，并且在两个驱动器上得到校验。

2) **稳定读 (stable read)**。稳定读首先从驱动器1上读取块。如果这一操作产生错误的ECC，则再次尝试读操作，一直到 n 次。如果所有这些操作都给出错误的ECC，则从驱动器2上读取对应的数据块。给定一个成功的稳定写为数据块留下两个可靠的副本这样的事实，并且我们假设在合理的时间间隔内相同的块在两个驱动器上自发地变坏的概率可以忽略不计，那么稳定读就总是成功的。

3) **崩溃恢复 (crash recovery)**。崩溃之后，恢复程序扫描两个磁盘，比较对应的块。如果一对块都是好的并且是相同的，就什么都不做。如果其中一个具有ECC错误，那么坏块就用对应的好块来覆盖。如果一对块都是好的但是不相同，那么就将驱动器1上的块写到驱动器2上。

如果不存在CPU崩溃，那么这一方法总是可行的，因为稳定写总是对每个块写下两个有效的副本，并且假设自发的错误决不会在相同的时刻发生在两个对应的块上。如果在稳定写期间出现CPU崩溃会怎么样？这就取决于崩溃发生的精确时间。有5种可能性，如图5-27所示。

3) 崩溃恢复 (crash recovery)。崩溃之后, 恢复程序扫描两个磁盘, 比较对应的块。如果一对块都是好的并且是相同的, 就什么都不做。如果其中一个具有ECC错误, 那么坏块就用对应的好块来覆盖。如果一对块都是好的但是不相同, 那么就将驱动器1上的块写到驱动器2上。

如果不存在CPU崩溃, 那么这一方法总是可行的, 因为稳定写总是对每个块写下两个有效的副本, 并且假设自发的错误决不会在相同的时刻发生在两个对应的块上。如果在稳定写期间出现CPU崩溃会怎么样? 这就取决于崩溃发生的精确时间。有5种可能性, 如图5-27所示。

在图5-27a中, CPU崩溃发生在写块的两个副本之前。在恢复的时候, 什么都不用修改而旧的值将继续存在, 这是允许的。

在图5-27b中, CPU崩溃发生在写驱动器1期间, 破坏了该块的内容。然而恢复程序能够检测出这一

错误, 并且从驱动器2恢复驱动器1上的块。因此, 这一崩溃的影响被消除并且旧的状态完全被恢复。

在图5-27c中, CPU崩溃发生在写完驱动器1之后但是还没有写驱动器2之前。此时已经过了无法复原的时刻: 恢复程序将块从驱动器1复制到驱动器2上。写是成功的。

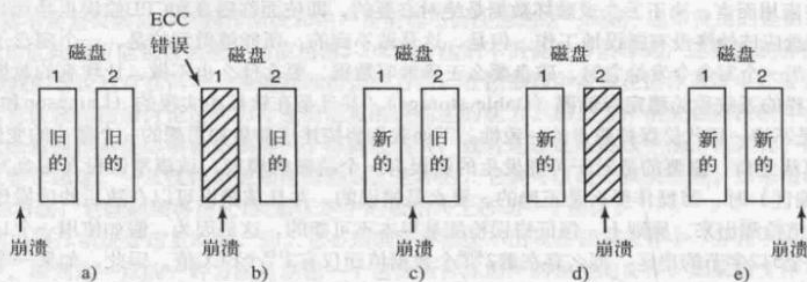


图5-27 崩溃对于稳定写的影响的分析

图5-27d与图5-27b相类似: 在恢复期间用好的块覆盖坏的块。不同的是, 两个块的最终取值都是新的。最后, 在图5-27e中, 恢复程序看到两个块是相同的, 所以什么都不用修改并且在此处写也是成功的。

对于这一模式进行各种各样的优化和改进都是可能的。首先, 在崩溃之后对所有的块两个两个地进行比较是可行的, 但是代价高昂。一个巨大的改进是在稳定写期间跟踪被写的是哪个块, 这样在恢复的时候必须被检验的块就只有一个。某些计算机拥有少量的非易失性RAM (nonvolatile RAM), 它是一个特殊的CMOS存储器, 由锂电池供电。这样的电池能够维持很多年, 甚至有可能是计算机的整个生命周

题目 1.5

如果一个程序为多个进程所共享, 那么该程序的代码在运行的过程中不能被修改, 即程序应是

答案

可重入程序 (或纯代码程序)

题目 1.6

死锁的四个必要条件是 _____、请求和保持条件即部分已分配、和

答案

互斥条件; 不剥夺条件; 循环等待条件

主机与磁盘间传递数据是以物理块为单位进行的。

答案

正确 (√)

题目 2.4

分段存储管理系统的地址空间都是二维的。

答案

正确 (√)

题目 2.7

文件目录的主要作用是按名存取。

答案

正确 (√)

题目 2.9

临界区是不可中断的程序。

答案

错误 (×)

题目 3.1

简述现代计算机为什么要区分两种运行状态 (管态和目态)?

答案

1. 保护系统资源：管态（内核态）允许执行特权指令（如控制 I/O、修改内存管理寄存器），目态（用户态）仅允许执行普通指令，防止用户程序非法操作硬件或修改系统核心数据，保障系统稳定性。
2. 隔离用户程序与系统程序：用户程序在目态运行，系统程序（操作系统内核）在管态运行，避免用户程序干扰系统程序的执行，确保操作系统的独立性和可靠性。
3. 简化系统设计：通过状态区分明确指令执行权限，减少权限判断的复杂度，使系统资源管理（如内存、设备）更有序。

解析

核心逻辑是“权限隔离与资源保护”：用户程序的行为不可控，区分状态可限制其权限，避免对系统造成破坏，同时让操作系统能独占核心资源的管理权限，确保系统正常运行。

题目 3.2

CPU 中的缓存和操作系统中的缓存的分别是什么？

答案

1. CPU 中的缓存（Cache）：
- 硬件层面的高速缓存，位于 CPU 与主存之间，分为 L1、L2、L3 等层级。

◦ 作用：缓解 CPU 与主存的速度差异，缓存 CPU 近期可能访问的指令和数据（基于局部性原理），减少主存访问次数，提高 CPU 执行效率。
2. 操作系统中的缓存（如页面缓存、文件缓存）：
- 软件层面的缓存，由操作系统管理，通常使用主存中的部分空间。

◦ 作用：缓存磁盘文件数据、页面数据等，减少磁盘 I/O 次数（如读取已缓存的文件无需访问磁盘），提高系统 I/O 效率。

解析

二者的核心区别是“层级与管理主体”：CPU 缓存是硬件管理的高速缓存，聚焦“CPU 与主存的速度匹配”；操作系统缓存是软件管理的主存缓存，聚焦“主存与外存的 I/O 效率”，均基于局部性原理设计。

题目 3.3

在网络编程中设计并发服务器，使用多进程与多线程有什么区别？

答案

对比维度	多进程并发服务器	多线程并发服务器
资源开销	高（进程有独立地址空间、PCB，创建 / 切换成本高）	低（线程共享进程地址空间，仅切换成本低）
通信方式	需借助进程间通信（IPC）机制（如管道、消息队列、共享内存），复杂	可直接共享进程的全局变量和堆，机制避免竞态）
稳定性	高（进程独立，一个进程崩溃不影响其他进程）	低（线程共享进程资源，一个线程崩溃）
扩展性	受进程数限制（系统进程数上限较低）	支持更多并发连接（线程数上限高）

解析

核心差异源于“进程与线程的资源隔离程度”：多进程通过独立资源保障稳定性，但牺牲开销和通信效率；多线程通过共享资源降低开销，提升并发量，但需额外处理同步问题，稳定性依赖线程安全设计。

题目 3.4

列举进程间互斥与同步的主要机制？

答案

1. 互斥机制（确保共享资源独占访问）：
 - 信号量（二元信号量，值为 0 或 1）；
 - 互斥锁（mutex，专门用于互斥的同步原语）；
 - 临界区（Windows 系统中的轻量级互斥机制）；
 - 原子操作（硬件支持的不可中断操作，如 Test-and-Set）。
2. 同步机制（协调进程执行顺序）：
 - 信号量（计数信号量，用于同步资源可用数量）；
 - 条件变量（与互斥锁配合，实现进程间的等待 - 唤醒）；
 - 管程（封装同步逻辑，简化同步代码）；
 - 消息传递（通过发送 / 接收消息协调执行顺序）。

题目 3.5

针对于已经很好地支持多进程的函数库，当引入线程机制时，原有进程中的“全局变量”会面临什么问题、如何解决？

答案

1. 面临的问题：
 - 全局变量被进程内所有线程共享，多个线程同时读写时会产生**竞态条件**，导致数据不一致（如全局计数器被多线程并发修改，结果出错）。
 - 原有函数库未考虑线程安全，可能因全局变量的非同步访问导致函数执行结果异常。
2. 解决方法：
 - 同步机制：**对全局变量的访问加锁**（如互斥锁、信号量），确保同一时刻只有一个线程读写。
 - **线程局部存储（TLS）**：将全局变量改为线程私有变量，每个线程拥有独立副本，避免共享冲突（如 POSIX 线程的 pthread_key_t）。
 - 重构代码：减少全局变量的使用，通过函数参数传递数据，避免共享状态。

解析

核心问题是“共享变量的并发访问冲突”，解决思路是“要么同步共享，要么取消共享”。同步机制保障共享变量的安全访问，线程局部存储则通过私有化变量彻底避免冲突，需根据场景选择。

题目 3.7

解决死锁问题常用哪几种措施？

答案

1. 死锁预防：破坏死锁的四个必要条件之一，从根本上避免死锁。
 - 破坏互斥条件：将独占资源改为共享资源（如 SPOOLing 技术虚拟打印机）；
 - 破坏请求和保持条件：进程启动时一次性申请所有资源，或释放已占资源后再申请新资源；
 - 破坏不剥夺条件：允许系统强制剥夺进程的闲置资源；
 - 破坏循环等待条件：对资源编号，进程按编号顺序申请资源。
2. 死锁避免：动态检查资源分配请求，避免系统进入不安全状态（如银行家算法）。
3. 死锁检测与恢复：允许死锁发生，通过算法检测死锁，再通过资源剥夺、进程终止、回滚等方式恢复系统。
4. 忽略死锁：适用于死锁概率极低的场景（如个人计算机），系统出现死锁时直接重启，简单高效。

题目 3.8

在现代计算机系统中，合理有效地使用存储器与否将直接影响到整个计算机系统作用的发挥，如何提高内存利用率？

答案

1. 内存分配优化：
 - 采用动态分区分配算法（如最佳适应、伙伴系统），减少外部碎片；
 - 内存紧凑（Compaction）：定期整理内存，将分散的空闲分区合并为连续大分区，分配给大作业。
2. 虚拟内存技术：
 - 采用请求分页 / 分段存储管理，仅将程序的部分页面 / 段装入内存，提高内存利用率（小内存运行大作业）；
 - 页面置换算法优化（如 LRU、老化算法），减少缺页中断，提升虚拟内存效率。
3. 资源共享：
 - 共享可重入程序（如系统库），多个进程共享同一份代码副本，减少内存占用；
 - 共享内存（IPC 机制），进程间通过共享内存传递数据，避免数据拷贝，节省内存。
4. 减少内存碎片：
 - 分页存储管理减少外部碎片（内部碎片可通过合理选择页大小优化）；
 - 分段分页结合（段页式），兼顾分段的逻辑独立性和分页的内存利用率。

解析

提高内存利用率的核心是“减少浪费（碎片）”“提高内存复用（共享）”“扩充逻辑内存（虚拟内存）”，通过硬件支持（如 MMU）和软件算法（如分配、置换算法）协同实现。

题目 3.9

比较 DMA 与通道的不同？

答案

对比维度	DMA（直接内存访问）	通道（I/O 通道）
本质	硬件控制器，直接在设备与内存间传输数据，无需 CPU 干预	独立的 I/O 处理器，可执行通道操作
功能范围	仅负责单个设备与内存的数据传输	可管理多个设备，执行复杂的 I/O 操作
控制方式	由 CPU 初始化传输参数（如地址、长度），传输过程自主完成	CPU 仅需启动通道程序，通道自果
灵活性	较低，仅支持特定设备的传输	较高，可通过通道程序适配不同
适用场景	高速设备（如磁盘、显卡）的批量数据传输	大型计算机系统，多设备并发 I/O

解析

二者的核心区别是“是否具备独立处理能力”：DMA 是“被动”的传输控制器，需 CPU 初始化；通道是“主动”的 I/O 处理器，可自主管理多个设备，灵活性和功能更强。

题目 2.3

简述操作系统的系统调用（或系统服务）实现原理及调用过程？

答案

1. 实现原理：系统调用是用户程序请求操作系统内核服务的接口，内核将复杂的硬件操作、资源管理等功能封装为系统调用，用户程序通过特定指令触发内核态切换，由内核执行特权操作后返回结果。
2. 调用过程：
 - 用户程序调用系统调用接口（如 C 语言的 `read()` 函数），将参数存入指定寄存器或栈。
 - 执行陷阱指令（如 `int 0x80`、`syscall`），触发 CPU 从用户态切换到内核态。
 - 内核根据系统调用号查找系统调用表，执行对应的内核服务函数（如处理文件读取、内存分配）。
 - 内核完成操作后，将结果存入指定位置，执行返回指令，切换回用户态。
 - 用户程序从系统调用接口返回，继续执行后续代码。

题目 2.4

针对于已经很好地支持多进程的函数库，当引入线程机制时，对于已有的不可重入库函数会面临什么问题、如何解决？

答案

1. 面临的问题：

- 不可重入库函数可能使用静态变量、全局变量或共享缓冲区，多个线程并发调用时会产生竞态条件，导致数据不一致（如函数内部的静态计数器被多线程同时修改）。
- 不可重入函数未考虑线程同步，可能因资源竞争导致函数执行结果异常、死锁等问题。

2. 解决方法：

- 同步机制：对函数的调用加锁（如互斥锁、信号量），确保同一时刻只有一个线程执行函数，避免竞态条件。
- 重构函数：将函数改为可重入函数，移除静态变量、全局变量，通过函数参数传递数据，使用线程局部存储（TLS）保存线程私有状态。
- 封装线程安全版本：为不可重入函数提供线程安全的封装接口，内部实现同步逻辑，不修改原函数代码。

解析

核心问题是“共享状态的并发访问冲突”，解决思路是“要么同步共享，要么取消共享”，需兼顾兼容性和线程安全性。

FAT32 文件系统的核心设计原理是 **“FAT 表管理簇链 + 层级目录 + 冗余保障”**，具体包括：

1. 以 32 位 FAT 表为核心，记录簇的分配状态和簇链关系，支持大容量分区和文件；
2. 采用簇作为基本存储单位，平衡存储效率与管理复杂度，文件数据通过簇链串联；
3. 分区结构固定（引导扇区 + FSINFO 扇区 + FAT 表 + 数据区），通过冗余 FAT 表和 FSINFO 扇区保障可靠性；
4. 以 32 字节目录项管理文件元信息，支持短 / 长文件名和丰富属性，根目录位于数据区，打破大小限制；
5. 结构简单、兼容性强，适配各类存储介质，通过簇链遍历实现文件的读写操作。

七、抖动 (Thrashing)

(一) 定义

抖动是指进程频繁发生页面故障，导致大部分时间都用于页面的换入换出（磁盘 I/O），而实际执行指令的时间很少的现象。此时，系统的 CPU 利用率极低，性能严重下降。

(二) 产生原因

1. **原因 1：多进程全局置换：**采用全局页面置换算法时，多个进程会相互抢占页框。当所有进程的工作集总大小超过物理内存容量时，每个进程的工作集无法全部驻留内存，导致频繁的页面故障和页框抢占，引发抖动。
2. **原因 2：多道程序度过高：**
 - 操作系统为了提高 CPU 利用率，不断增加多道程序度，引入更多进程。
 - 新进程需要页框，会置换现有进程的页框，导致现有进程页面故障率上升。
 - 现有进程为了维持自己的工作集，又会置换其他进程的页框，形成恶性循环，最终所有进程都陷入频繁的页面换入换出，CPU 利用率急剧下降。

(三) 预防与解决方法

1. **采用局部置换算法：**每个进程的页框仅在自己的池中置换，避免进程间相互抢占页框，减少抖动的可能性。
2. **工作集模型：**确保每个进程的工作集全部驻留内存。当系统总工作集大小超过物理内存时，减少多道程序度，换出部分进程，释放页框。
3. **页面故障频率 (PFF) 控制：**
 - 当进程 PFF 过高时，为其增加页框；当 PFF 过低时，减少其页框。
 - 当系统整体 PFF 过高时，减少多道程序度，避免抖动。
4. **合理设置页面大小：**根据系统中进程的平均大小，选择最优页面大小，平衡内部碎片和页表开销，减少页面故障率。
5. **使用进程优先级：**为重要进程分配更多页框，确保其工作集驻留内存，避免被其他进程抢占页框。

题目 1.1

虚拟存储管理系统的基础是程序的_____理论。

答案

局部性

1. 内存利用率低的体现：

- **碎片问题：**包括内部碎片（分页存储中页内未使用的空间）和外部碎片（动态分区中分散的空闲空间）；
- **内存空闲：**内存中存在大量未被利用的空闲空间（例如单道程序运行时，内存仅被一个程序占用）；
- **程序未充分加载：**程序全部装入内存，但仅部分代码 / 数据被访问（违背局部性原理）。

2. 提升内存利用率的方法：

- **采用虚拟存储技术：**基于局部性原理，仅将程序的部分页面 / 段装入内存（如请求分页）；
- **优化内存分配算法：**采用动态分区分配（如伙伴系统）减少外部碎片，分页存储减少外部碎片；
- **内存紧凑：**定期整理内存，合并分散的空闲分区；
- **共享资源：**共享可重入程序（如系统库），多个进程共享同一份代码副本。

2、添加系统调用

(1) 声明系统调用原型

vim include/linux/syscalls.h

```
asmlinkage long sys_schello(void);  
#endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */
```

(2) 实现系统调用功能

vim kernel/sys.c

```
SYSCALL_DEFINE0(gettid)  
{  
    return task_pid_vnr(current);  
}  
  
SYSCALL_DEFINE0(schello)  
{  
    printk("Hello new system call schello! hello LZH2311451\n");  
    return 0;  
}
```

(3) 注册系统调用

vim arch/x86/entry/syscalls/syscall_64.tbl

466	common	removexattrat	sys_removexattrat
467	common	open_tree_attr	sys_open_tree_attr
468	common	schello	sys_schello

(4) 编译内核

只需增量编译，无需 make clean

直接 make -j8

(5) 安装内核模块

sudo make modules_install

(6) 安装内核

sudo make install

操作系统的发展动力是硬件迭代、应用拓展、用户需求、理论创新、安全诉求与产业政策等多方面的协同合力：硬件技术（如处理器、存储、设备形态）的突破为其提供底层支撑，推动 OS 适配并行算力、大容量存储与多样化设备；应用场景从单机计算向分布式、云原生、物联网的拓展，倒逼 OS 突破功能边界，提供虚拟化、实时调度等能力；用户对高效、便捷体验的追求驱动 OS 优化交互设计与资源利用率；并发同步、调度算法等理论创新与虚拟化、分布式等技术突破为其提供实现路径；安全可靠的刚性需求推动 OS 完善权限隔离、漏洞防护等机制；而开源生态协同与国家信创等政策导向，则进一步加速 OS 的技术迭代与产业化落地，共同推动操作系统持续演进。

王道论坛 bilibili

Swap指令

有的地方也叫 Exchange 指令，或简称 XCHG 指令。

Swap 指令是用硬件实现的，执行的过程不允许被中断，只能一气呵成。以下是用C语言描述的逻辑

```
//Swap 指令的作用是交换两个变量的值
Swap (bool *a, bool *b) {
    bool temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

```
//以下是用 Swap 指令实现互斥的算法逻辑
//lock 表示当前临界区是否被加锁
bool old = true;
while (old == true)
    Swap (&lock, &old);
临界区代码段...
lock = false;
剩余区代码段...
```

逻辑上来看 Swap 和 TSL 并无太大区别，都是先记录下此时临界区是否已经被上锁（记录在 old 变量上），再将上锁标记 lock 设置为 true，最后检查 old，如果 old 为 false 则说明之前没有别的进程对临界区上锁，则可跳出循环，进入临界区。

王道考研/CSKAOYAN.COM

王道论坛 bilibili

TestAndSet指令

简称 TS 指令，也有地方称为 TestAndSetLock 指令，或 TSL 指令

TSL 指令是用硬件实现的，执行的过程不允许被中断，只能一气呵成。以下是用C语言描述的逻辑

```
//布尔型共享变量 lock 表示当前临界区是否被加锁
//true 表示已加锁, false 表示未加锁
bool TestAndSet (bool *lock){
    bool old;
    old = *lock; //old用来存放lock 原来的值
    *lock = true; //无论之前是否已加锁, 都将lock设为true
    return old; //返回lock原来的值
}
```

```
//以下是使用 TSL 指令实现互斥的算法逻辑
while (TestAndSet (&lock)); //“上锁”并“检查”
临界区代码段...
lock = false; //“解锁”
剩余区代码段...
```

若刚开始 lock 是 false，则 TSL 返回的 old 值为 false，while 循环条件不满足，直接跳过循环，进入临界区。若刚开始 lock 是 true，则执行 TSL 后 old 返回的值为 true，while 循环条件满足，会一直循环，直到当前访问临界区的进程在退出区进行“解锁”。相比软件实现方法，TSL 指令把“上锁”和“检查”操作作用硬件的方式变成了一气呵成的原子操作。优点：实现简单，无需像软件实现方法那样严格检查是否有逻辑漏洞；适用于多处理机环境

王道考研/CSKAOYAN.COM

文件管理系统设计与评价

一、文件管理系统整体设计框架

本文件管理系统基于分层架构设计，核心目标是实现文件 / 目录的高效管理、数据可靠性存储、资源合理分配及用户便捷交互，支持单机场景下的多用户文件操作，兼容常见文件类型与访问模式。

二、对外接口（API）设计

对外提供简洁统一的系统调用接口，覆盖文件与目录全生命周期管理，支持设备独立性与多用户权限控制：

接口名称	功能描述	参数说明
<code>create_file(path, mode)</code>	创建普通文件	<code>path</code> ：文件绝对 / 相对路径； <code>mode</code> ：权限标识（如 <code>0755</code> ，表示所有者可读写，组用户可执行组合）
<code>create_dir(path)</code>	创建目录	<code>path</code> ：目录路径
<code>open(path, flag)</code>	打开文件 / 目录	<code>path</code> ：目标路径； <code>flag</code> ：打开模式（只读 / 读写 / 追加等）
<code>close(fd)</code>	关闭文件 / 目录	<code>fd</code> ：文件描述符
<code>read(fd, buf, len)</code>	读取文件数据	<code>fd</code> ：文件描述符； <code>buf</code> ：数据缓冲区； <code>len</code> ：读取长度
<code>write(fd, buf, len)</code>	写入文件数据	<code>fd</code> ：文件描述符； <code>buf</code> ：待写数据； <code>len</code> ：写入长度
<code>delete(path)</code>	删除文件 / 目录	<code>path</code> ：目标路径
<code>rename(old_path, new_path)</code>	重命名文件 / 目录	<code>old_path</code> ：原路径； <code>new_path</code> ：新路径
<code>get_attr(path)</code>	获取文件 / 目录属性	<code>path</code> ：目标路径
<code>set_attr(path, attr)</code>	修改文件 / 目录属性	<code>path</code> ：目标路径； <code>attr</code> ：待修改属性
<code>list_dir(path)</code>	列出目录内容	<code>path</code> ：目录路径

三、文件和目录的物理实现

1. 存储介质与块划分

- 采用磁盘作为存储介质，将磁盘划分为固定大小的物理块（默认 4KB），块是存储分配的基本单位。
- 磁盘布局分为：MBR（主引导记录）、引导块、超级块、inode 区、数据区、空闲区，其中超级块存储文件系统整体信息（块总数、空闲块数、inode 总数等）。

2. 文件的物理实现

- 采用**inode 索引节点机制**管理文件元信息与数据块映射：
 - inode 结构：包含文件权限、所有者、创建时间、修改时间、大小、数据块指针等属性，每个文件对应唯一 inode。
 - 数据块映射：采用“直接索引 + 间接索引”混合方式，inode 中前 12 个指针为直接索引（指向数据块），第 13 个为一级间接索引（指向索引块，索引块存储数据块指针），第 14 个为二级间接索引，第 15 个为三级间接索引，支持最大文件大小达 4GB（4KB 块大小）。
- 文件逻辑结构：支持字节流（普通文件）、记录序列（日志文件）两种类型，满足不同应用场景需求。

3. 目录的物理实现

- 目录本质是特殊文件，每个目录对应一个 inode，其数据块存储目录项列表。
- 目录项结构（32 字节）：包含文件名（14 字节）、inode 编号（4 字节）、文件类型标识（1 字节）、保留字段（13 字节），支持快速查找文件名与 inode 的映射关系。
- 目录组织结构：采用树形结构，根目录 inode 固定编号为 2，每个目录项包含父目录指针与子目录 / 文件指针，支持绝对路径与相对路径解析。

四、空闲区的管理

采用**位图 + 成组链接法**结合的方式管理空闲块，兼顾分配效率与空间利用率：

1. **位图管理**：用 1 位标识一个物理块的状态（0 表示空闲，1 表示占用），位图存储在超级块附近的连续块中，支持快速查询空闲块位置。
2. **成组链接法**：将空闲块按组划分，每组包含若干空闲块（默认 256 个），每组的第一个块存储下一组空闲块的起始地址与块数，最后一组的链接地址设为 - 1（表示结束）。
3. **分配流程**：分配时先从当前组获取空闲块，若当前组无空闲块，通过链接地址查找下一组；若位图指示无空闲块，返回分配失败。
4. **回收流程**：回收块时将其加入当前组，若当前组已满则新建一组，更新位图与组链接信息，确保空闲块快速复用。

五、其他核心部分

1. 权限与安全管理

- 采用“用户 - 组 - 其他”三级权限模型，每个文件 / 目录的 inode 中存储权限位（读 r、写 w、执行 x），控制不同用户的访问权限。
- 支持文件上锁机制（通过 `lock()` / `unlock()` 接口），避免多进程并发访问导致的竞态条件，保障数据一致性。

2. 缓存机制

- 实现块缓存（Buffer Cache），将频繁访问的磁盘块缓存到内存中，减少磁盘 I/O 次数，提升读写性能。
- 采用 LRU（最近最少使用）算法淘汰缓存块，缓存大小可动态调整（默认占物理内存的 20%）。

3. 可靠性保障

- 日志机制：关键操作（如 inode 修改、目录项更新、空闲块分配）先写入日志区，成功后再同步到数据区，避免系统崩溃导致的数据不一致。
- 数据备份：支持定期增量备份与全量备份，备份数据存储在独立磁盘分区或外部存储介质。

4. 目录查找优化

- 采用哈希表加速目录查找，每个目录的目录项在内存中构建哈希表，根据文件名哈希值快速定位目录项，查找时间复杂度优化为 $O(1)$ 。

六、系统评价

1. 优势

- 功能完备：覆盖文件 / 目录的全生命周期操作，支持多用户权限控制与并发访问，满足日常使用需求。
- 性能高效：inode 索引机制支持大文件存储，位图 + 成组链接法提升空闲块分配与回收效率，块缓存与哈希目录查找减少 I/O 开销。
- 可靠性高：日志机制与备份功能保障数据一致性，避免系统崩溃或硬件故障导致的数据丢失。
- 扩展性强：物理块大小、inode 结构、缓存策略均可配置，支持后续扩展为网络文件系统（如 NFS）或分布式文件系统。

2. 不足

- 空间开销：inode 区占用固定磁盘空间，若小文件数量过多，可能导致 inode 耗尽而磁盘仍有空闲空间。
- 碎片问题：长期频繁的文件创建与删除会产生磁盘碎片，影响连续数据块的访问性能（需定期碎片整理）。
- 并发性能上限：单磁盘 I/O 瓶颈限制了高并发场景下的吞吐量，不适合大规模分布式场景。
- 日志开销：日志同步增加了部分操作的延迟，在高频小文件写入场景下性能损耗较明显。

题目 2.1

下列选项中，_____不是一个操作系统环境。

A. 赛扬 (Celeron) B. Linux C. Windows CE D. Solaris

答案

A

解析

- 选项 A：赛扬 (Celeron) 是英特尔推出的 CPU 型号，属于计算机硬件，并非操作系统；
 - 选项 B：Linux 是开源类 UNIX 操作系统，广泛用于服务器、桌面设备；
 - 选项 C：Windows CE 是微软针对嵌入式设备的操作系统；
 - 选项 D：Solaris 是 Sun 公司（现属 Oracle）的 UNIX 类操作系统。
- 因此答案为 A。

题目 2.2

为使进程从阻塞状态变为就绪状态，应利用_____原语？

A. BLOCK B. WAKEUP C. SUSPEND D. ACTIVE

答案

B

解析

- 原语功能对应：
 - BLOCK（阻塞）原语：使进程从运行态→阻塞态（因等待资源主动放弃 CPU）；
 - WAKEUP（唤醒）原语：使进程从阻塞态→就绪态（等待的资源就绪，触发唤醒）；
 - SUSPEND（挂起）原语：使进程从运行态 / 就绪态→挂起态（被动暂停）；
 - ACTIVE（激活）原语：使进程从挂起态→就绪态（解除挂起）。
- 题目要求“阻塞→就绪”，因此答案为 B。

5.7 用户界面 (User Interfaces)

用户界面是用户与操作系统交互的桥梁，分为命令行界面和图形用户界面，由 I/O 软件的用户层实现。

5.7.1 输入软件

1. 键盘软件：

- 工作原理：键盘按键按下时，产生扫描码，键盘控制器将扫描码转换为 ASCII 码或 Unicode 码，通过中断发送给 CPU；键盘软件处理扫描码，支持按键组合（如 Ctrl+C、Shift+A）、字符编码转换等。
- 工作模式：
 - 规范模式 (Canonical Mode)：也称行缓冲模式，用户输入一行字符后，按回车才将数据发送给应用程序，支持回显、删除、换行等功能，适用于命令行界面。
 - 非规范模式 (Non-canonical Mode)：也称无缓冲模式，用户按下按键后立即将数据发送给应用程序，无回显，适用于实时交互场景（如游戏、编辑器）。

2. 鼠标软件：

- 工作原理：鼠标移动或点击时，产生位移量和按键状态信息，通过中断发送给 CPU；鼠标软件处理这些信息，转换为屏幕坐标和点击事件（如左键单击、右键双击），发送给应用程序。
- 支持功能：鼠标指针移动、拖拽、滚轮滚动、多按键操作。

5.7.2 输出软件

1. **文本窗口 (Text Windows)**：命令行界面的输出方式，将字符按行和列显示在屏幕上，支持字符颜色、光标移动、清屏等功能，如 Linux 的 Bash 终端、Windows 的 CMD 终端。
2. **图形用户界面 (GUI, Graphical User Interface)**：通过图形元素（窗口、图标、菜单、按钮）实现交互，直观易用，是现代操作系统的主流界面。
 - 核心组件：
 - 窗口管理器：管理窗口的创建、移动、大小调整、关闭，如 Linux 的 GNOME、KDE，Windows 的 Explorer。
 - 图形设备接口 (GDI)：提供图形绘制功能（如绘制直线、矩形、图像），如 Windows 的 GDI+、Linux 的 X11。
 - 示例：
 - X Window System：Linux、Unix 系统的图形界面基础，采用客户端 - 服务器架构，支持远程图形界面。
 - Microsoft Windows：集成图形界面，支持窗口、菜单、控件等元素，提供丰富的图形 API。
3. **瘦客户端 (Thin Clients)**：仅提供用户界面功能，依赖服务器进行数据处理和存储，适用于企业办公、云桌面等场景，优点是硬件成本低、易管理；缺点是依赖网络，离线无法使用。

5.8.1 硬件相关问题

- **电池类型：**

- 一次性电池：如干电池，适用于低功耗设备，不可充电。
- 可充电电池：如锂电池、镍氢电池，适用于移动设备，支持重复充电，操作系统需管理电池充电和放电状态。

- **耗电硬件组件：**CPU、GPU、内存、硬盘、显示器、无线网卡（Wi-Fi、蓝牙）、传感器等，这些组件的能耗占设备总能耗的绝大部分。

5.8.2 操作系统相关问题

1. 组件电源管理：

- CPU 调频：根据系统负载调整 CPU 频率，轻负载时降低频率，减少能耗；重负载时提高频率，保证性能（如 Intel 的 SpeedStep、AMD 的 Cool'n'Quiet）。
- 显示器管理：闲置时关闭显示器或降低亮度，减少能耗。
- 硬盘管理：闲置时停止硬盘旋转（如笔记本电脑的硬盘休眠），减少能耗。
- 无线通信管理：闲置时关闭 Wi-Fi、蓝牙，或降低发射功率。

2. **热管理 (Thermal Management)**：

监测硬件温度，当温度过高时，降低 CPU 频率、启动风扇散热，避免硬件损坏；温度较低时，提高 CPU 频率、降低风扇转速，平衡性能和能耗。

3. **电池管理：**

监测电池电量、电压、温度，估算剩余电量和使用时间；管理充电过程，避免过充、过放，延长电池寿命；支持省电模式（如低电量时关闭非必要功能）。

4. 电源管理接口：

- ACPI (Advanced Configuration and Power Interface)：高级配置和电源管理接口，是操作系统与硬件之间的电源管理标准，支持电源状态切换（如开机、休眠、待机、关机）、设备电源控制、热管理等功能。

5.8.3 应用程序相关问题

- 应用程序需适配电源管理策略，如闲置时降低运行频率、减少网络请求和磁盘 I/O，避免不必要的能耗；低电量时提示用户保存数据，关闭非核心功能。
- 操作系统提供电源管理 API，应用程序可通过 API 获取电池状态、设置电源使用策略（如高性能、省电模式）。



类 Unix

FreeBSD, OpenBSD, NetBSD, Linux, MINIX, macOS

Linux

Ubuntu, CentOS, Debian, Android, 银河麒麟, Deepin

国产

鸿蒙, 银河麒麟, 中标麒麟, Deepin, 统信 UOS

最优置换算法 (Optimal Replacement Algorithm)

未来最长时间内不会被访问

先进先出置换算法 (FIFO Replacement Algorithm)

将页面按加载顺序组织成一个队列，发生页面故障时，置换队列头部（最早进入）的页面

最近未使用置换算法 (NRU, Not Recently Used)

“访问位 (R)” 和 “修改位 (M)” 优先置换优先级最低的页面

二次机会置换算法 (Second Chance Replacement Algorithm)

FIFO 算法的改进

它为每个页面增加了一个访问位 R

时钟置换算法 (Clock Replacement Algorithm)

二次机会算法的高效实现

当发生页面故障时，从指针当前位置开始检查

最近最少使用置换算法 (LRU, Least Recently Used)

维护一个双向链表，页面被访问时，将其移到链表头部；发生页面故障时，置换链表尾部的页面（最久未访问）

未经常使用置换算法 (NFU, Not Frequently Used)

通过计数器记录页面的访问频率，每个时钟中断，操作系统遍历所有页面，将页面的访问位 R (0 或 1) 加到计数器中，然后将 R 置为 0

老化算法 (Aging Algorithm)

NFU 的改进，通过 “移位寄存器” 记录页面的近期访问历史，解决了 NFU 计数

器累积历史的问题。

为每个页面分配一个 n 位（如 8 位）的移位寄存器

每个时钟中断，将页面的访问位 R 移到移位寄存器的最高位，其余位向右移位，最低位丢弃；然后将 R 置为 0。

最少使用置换算法（LFU, Least Frequently Used）

与老化算法类似，但它仅关注最近 n 个时间周期内的访问频率，而非累积历史

计算移位寄存器中 1 的个数

工作集置换算法（Working-Set Replacement Algorithm）